

Laboratorios:

<https://developer.android.com/get-started/codelabs?hl=es-419>

Diseños básicos en Compose

54 minutos

Updated 20 de abril de 2026

En este codelab, aprenderás a implementar diseños reales con elementos componibles y modificadores que Compose proporciona de forma predeterminada.

1. Introducción

Como es un kit de herramientas de IU, Compose facilita la implementación de los diseños de tu app. Tú describes el modo en que quieres que se vea la IU y Compose se encargará de dibujarla en la pantalla. En este codelab, aprenderás a escribir IU de Compose. Se supone que comprendes los conceptos que se enseñan en el [codelab de aspectos básicos](#), así que primero asegúrate de completarlo. En ese codelab, aprendiste a implementar diseños simples con Surfaces, Rows y Columns. También aumentaste estos diseños con modificadores como padding, fillMaxWidth y size.

En este codelab, implementarás un **diseño más realista y complejo**, y durante el proceso aprenderás sobre varios **elementos componibles listos para usar y modificadores**. Después de finalizar este codelab, deberías poder transformar el diseño básico de una app en un código que funcione.

Este codelab no agrega ningún comportamiento real a la app. Si deseas conocer sobre el estado y la interacción, completa el [codelab de Estado en Compose](#).

Para obtener más asistencia mientras realizas este codelab, consulta el siguiente código:

Qué aprenderás

En este codelab, aprenderás lo siguiente:

- Cómo te ayudan los modificadores a aumentar tus elementos componibles
- Cómo los componentes de diseño estándar, como Column y LazyRow, posicionan elementos secundarios componibles
- Cómo los alineamientos y los arreglos cambian la posición de los elementos secundarios componibles en su elemento superior
- Cómo los elementos de Material componibles, como Scaffold y Bottom Navigation, te ayudan a crear diseños comprensivos
- Cómo compilar elementos componibles flexibles con las APIs de ranuras
- Cómo compilar diseños para diferentes configuraciones de pantalla

Qué necesitarás

- [Versión más reciente de Android Studio](#).
- Tener experiencia con la sintaxis de Kotlin, incluidas las funciones de lambdas.
- Tener experiencia básica con Compose (si aún no lo hiciste, completa el [codelab de los principios básicos de Jetpack Compose](#) antes de comenzar este codelab)
- Conocimientos básicos sobre lo que es un elemento componible y qué son los modificadores

Qué compilarás

En este codelab, implementarás un diseño realista de apps basado en simulaciones proporcionadas por un diseñador. **MySoothe** es una app de bienestar que enumera varias formas de mejorar tu cuerpo y mente. Contiene una sección que enumera tus colecciones favoritas y una sección con ejercicios físicos. Así se ve la app:

12:00



 Search

Align your Body



Inversions



Quick yoga



Stretching



Tabata

Favorite Collections



Short mantras



Stre



Nature meditations



Self



Home



Profile

[2. Prepárate](#)

En este paso, descargarás código que contiene temas y algunas configuraciones básicas.

Obtén el código

El código de este codelab se puede encontrar en el [repositorio de GitHub de codelab-android-compose](#). Para clonarlo, ejecuta lo siguiente:

```
$ git clone https://github.com/android/codelab-android-compose
```

Como alternativa, puedes descargar dos archivos ZIP:

- [Punto de partida](#)
- [Solución](#)

Consulta el código

El código descargado contiene el código de todos los codelabs de Compose disponibles. Para completar este codelab, abre el proyecto BasicLayoutsCodelab en Android Studio.

El repositorio compose-codelabs contiene código de partida para todos los codelabs en la ruta de aprendizaje.

En este codelab, usa el proyecto **BasicLayoutsCodelab**.

- **BasicLayoutsCodelab:** Es un proyecto que contiene el código de inicio y el código finalizado para este codelab.

El proyecto tiene componentes en diferentes ramas de git:

- **main:** Contiene el código de partida para este proyecto. Haz tus cambios aquí para completar el codelab.
- **end:** Contiene la solución para este codelab.

Te recomendamos que comiences con el código de la rama main y sigas el codelab paso a paso a tu propio ritmo.

[3. Comienza con un plan](#)

Para comenzar, implementaremos el diseño vertical de la app. Veamos esto con más detalle:



Search

Align your Body



Inversions



Quick yoga



Stretching



Tabata

Favorite Collections



Short mantras



Nature meditations



Home



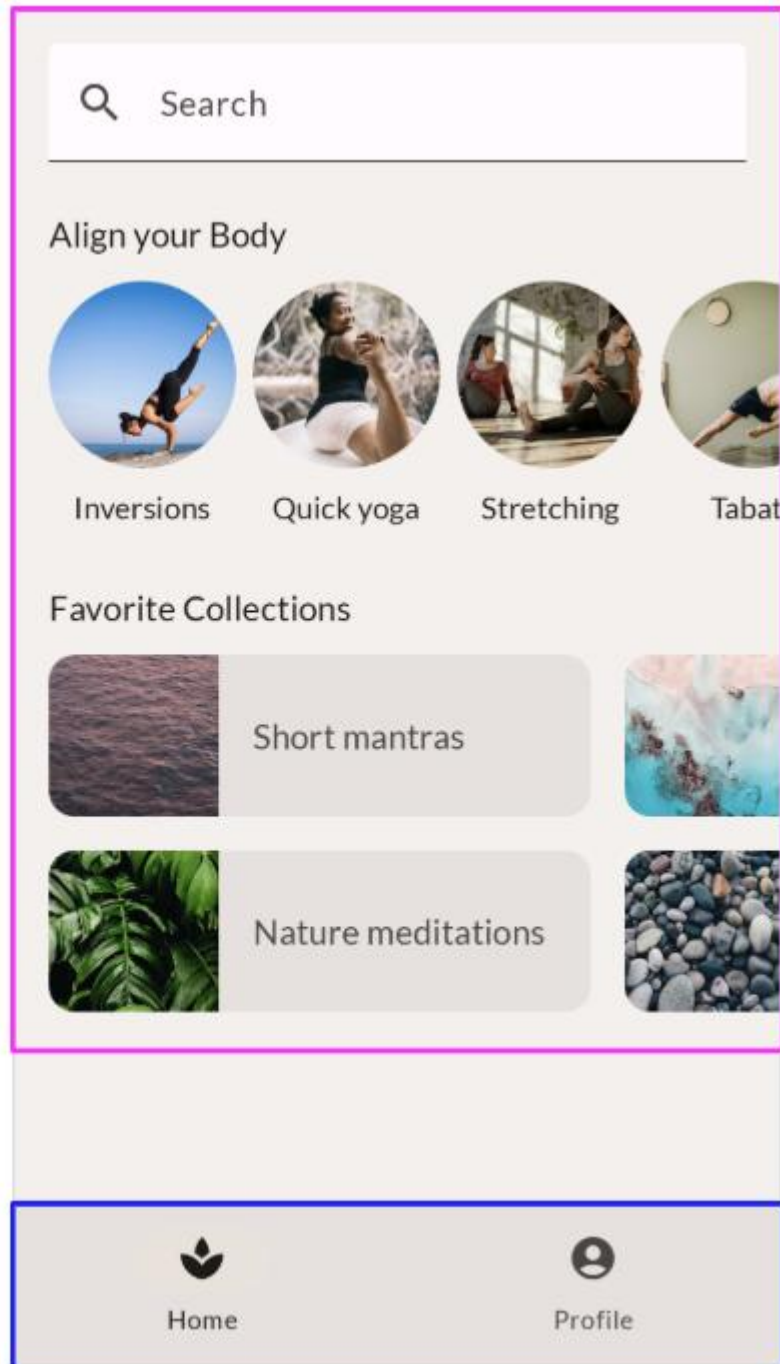
Profile

Cuando se te solicita que implementes un diseño, una buena manera de comenzar es entender claramente su estructura. No comiences a programar de inmediato, en cambio, **analiza el diseño en sí mismo**. ¿Cómo puedes **dividir la IU en varias partes reutilizables**?

Pongamos esto a prueba con nuestro diseño. En el nivel de abstracción más alto, podemos dividir este diseño en las siguientes dos partes:

- El contenido de la pantalla
- La barra de navegación inferior

Screen content



Bottom navigation

Cuando se desglosa, la pantalla contiene las siguientes tres subpartes:

- La Barra de búsqueda
- Una sección que se llama "Align your body"
- Una sección que se llama "Favorite collections"

**Search
bar**

 Search

**Align your
body section**

Align your Body



Inversions



Quick yoga



Stretching



Tabata

**Favorite
collections
section**

Favorite Collections



Short mantras

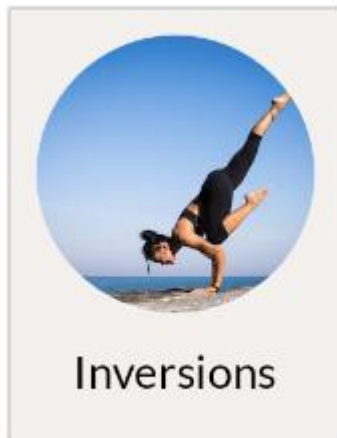


Nature meditations

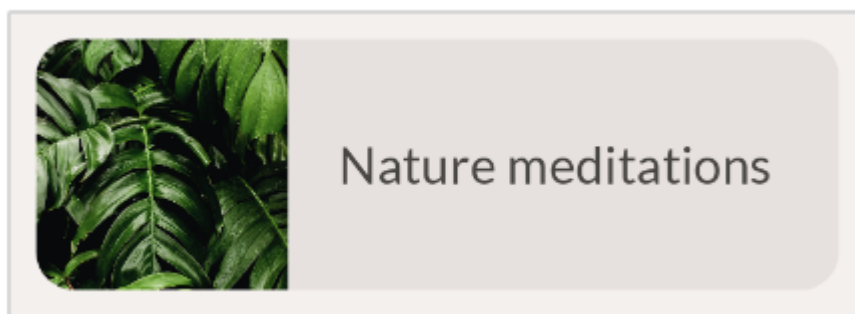


Dentro de cada sección, también puedes ver algunos componentes de nivel inferior que se volvieron a usar, como los siguientes:

- El elemento "Align your body" que se muestra en una fila desplazable horizontalmente



- La tarjeta "Favorite collection" que se muestra en una cuadrícula desplazable horizontalmente



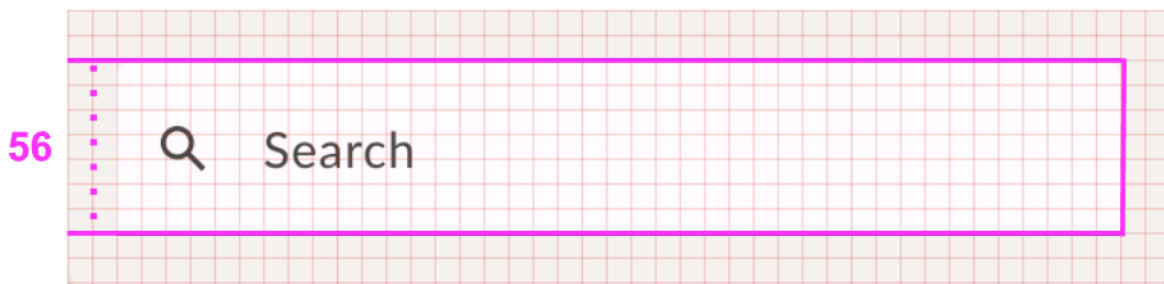
Ahora que analizaste el diseño, puedes comenzar a implementar elementos componibles de cada parte identificada de la IU. Comienza con los elementos componibles de nivel más bajo y combínalos en otros más complejos. Al final del codelab, tu nueva app se verá como el diseño proporcionado.

[4. Barra de búsqueda: Modificadores](#)

El primer elemento que se transforma en uno componible es la barra de búsqueda. Volvamos a observar el diseño:



En función de esta única captura de pantalla, sería bastante difícil implementar este diseño de manera perfecta en términos de píxeles. En general, un diseñador transmite más información sobre su trabajo. Puede darte acceso a su herramienta de diseño o compartir los llamados diseños de revisión. En este caso, nuestro diseñador transfirió los diseños de revisión, que puedes usar para identificar cualquier valor de tamaño. El diseño se muestra con una superposición de cuadrícula de 8 dp de modo que puedas ver con facilidad el espacio entre los elementos y a su alrededor. Además, se agregan algunos espaciados de forma explícita para aclarar determinados tamaños.



Puedes ver que la barra de búsqueda debe tener una altura de 56 píxeles independientes de la densidad. También debe llenar todo el ancho de su elemento superior.

Para implementar la barra de búsqueda, usa un componente de Material llamado [Campo de texto](#). La biblioteca de Compose Material contiene un elemento componible llamado [TextField](#), que es la implementación de este componente de Material.

Comienza con una implementación básica de `TextField`. En tu base de código, abre `MainActivity.kt` y busca el elemento `SearchBar` componible.

Dentro del elemento llamado `SearchBar`, escribe la implementación básica de `TextField`:

```
import androidx.compose.material3.TextField
```

```
@Composable  
fun SearchBar(  

```

```

        modifier: Modifier = Modifier
    ){
        TextField(
            value = "",
            onChange = {},
            modifier = modifier
        )
    }
}

```

Estos son algunos puntos que deberías tener en cuenta:

- Codificaste el valor del campo de texto, y la devolución de llamada `onChange` no hace nada. Como este es un codelab que se centra en el diseño, ignora cualquier tarea relacionada con el estado.

Si deseas obtener más información sobre el estado de los elementos que admiten composición, consulta el [codelab de Estado en Compose](#).

- La función de componibilidad `SearchBar` acepta un parámetro `modifier` y lo pasa al `TextField`. Esta es una práctica recomendada de acuerdo con los [lineamientos de Compose](#). Esto permite que el llamador del método modifique el aspecto del elemento que admite composición, lo que lo hace más flexible y reutilizable. Seguirás con esta práctica recomendada para todos los elementos componibles en este codelab.

Veamos la vista previa de este elemento componible. Recuerda que puedes usar la [funcionalidad de vista previa](#) en Android Studio para iterar rápidamente en elementos componibles individuales. `MainActivity.kt` contiene vistas previas de todos los elementos de este tipo que compilarás en este codelab. En este caso, el método `SearchBarPreview` procesa nuestro elemento `SearchBar` que admite composición, con cierto fondo y padding para darle un poco más de contexto. Luego de la implementación que acabas de agregar, debería verse de la siguiente manera:

SearchBarPreview



Faltan algunas funciones. Primero, corrijamos el tamaño del elemento componible con los [modificadores](#).

Cuando escribes elementos componibles, usas **modificadores** para lo siguiente:

- Cambiar el tamaño, el diseño, el comportamiento y el aspecto del elemento que admite composición
- Agregar información (p. ej., etiquetas de accesibilidad)
- Procesar entradas del usuario
- Agregar interacciones de nivel superior, (p. ej., hacer que un elemento sea apto para hacer clic, desplazable, arrastrable o ampliable)

Cada elemento componible que llamas tiene un parámetro modifier que puedes configurar para adaptar su apariencia y comportamiento. Cuando estableces el modificador, puedes encadenar varios métodos de modificación para crear una adaptación más compleja.

Si quieres obtener más información sobre el comportamiento de los modificadores, consulta la [documentación sobre los modificadores](#). También puedes consultar la [lista completa de modificadores disponibles](#).

En este caso, la barra de búsqueda debe medir al menos 56 dp y llenar el ancho de su elemento superior. A fin de encontrar los modificadores adecuados, consulta la [lista de modificadores](#) y la [sección de Size \(tamaño\)](#). Para la altura, puedes usar el modificador [heightIn](#). De esta manera, se garantiza que el elemento componible tenga una altura mínima específica. Sin embargo, podrá aumentar de tamaño cuando, por ejemplo, el usuario aumente el tamaño de la fuente del sistema. Para el ancho, puedes usar el modificador [fillMaxWidth](#). Este modificador hace que la barra de búsqueda ocupe todo el espacio horizontal de su elemento superior.

Actualiza el modificador de modo que coincida con el siguiente código:

```
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.heightIn
import androidx.compose.material3.TextField
```

```
@Composable
fun SearchBar(
    modifier: Modifier = Modifier
){
    TextField(
        value = "",
        onChange = {},
        modifier = modifier
            .fillMaxWidth()
            .heightIn(min = 56.dp)
    )
}
```

En este caso, debido a que un modificador influye en el ancho y el otro en la altura, no importa el orden de estos modificadores.

También debes configurar algunos parámetros de `TextField`. Intenta hacer que el elemento componible se vea como el diseño configurando los valores de los parámetros. Una vez más, este es el diseño de referencia:



Estos son los pasos que debes seguir para actualizar tu implementación:

- Agrega el ícono de búsqueda. `TextField` contiene un parámetro `leadingIcon` que acepta otro elemento componible. Dentro, puedes establecer un `Icon`, que en nuestro caso debería ser el ícono `Search`. Asegúrate de usar la importación de `Icon` correcta de `Compose`.

- Puedes usar `TextFieldDefaults.colors` para anular colores específicos. Establece los elementos `focusedContainerColor` y `unfocusedContainerColor` del campo de texto en el color `surface` de `MaterialTheme`.
- Agrega un texto de marcador de posición "Search" (puedes encontrarlo como el recurso de cadenas `R.string.placeholder_search`).

Cuando termines, el elemento componible debería tener un aspecto similar al siguiente:

```
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.heightIn
import androidx.compose.ui.res.stringResource
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.material3.TextField
import androidx.compose.material3.TextFieldDefaults
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Search
```

```
@Composable
fun SearchBar(
    modifier: Modifier = Modifier
){
    TextField(
        value = "",
        onChange = {},
        leadingIcon = {
            Icon(
                imageVector = Icons.Default.Search,
                contentDescription = null
            )
        },
        colors = TextFieldDefaults.colors(
            unfocusedContainerColor = MaterialTheme.colorScheme.surface,
            focusedContainerColor = MaterialTheme.colorScheme.surface
        ),
```

```
placeholder = {
    Text(stringResource(R.string.placeholder_search))
},
modifier = modifier
    .fillMaxWidth()
    .heightIn(min = 56.dp)
)
}
```

Observa lo siguiente:

- Agregaste un `leadingIcon` que muestra el ícono de búsqueda. Este ícono no necesita una descripción de contenido porque el marcador de posición del campo de texto ya describe su significado. Recuerda que una descripción de contenido en general se usa para fines de accesibilidad y proporciona al usuario de tu app una representación textual de una imagen o un ícono.

Si deseas obtener más información sobre la accesibilidad, consulta el [codelab sobre ese tema](#) o lee la [documentación sobre accesibilidad](#).

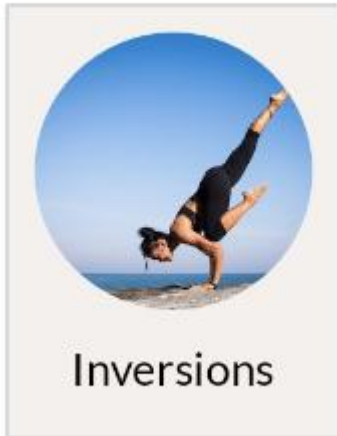
- A fin de adaptar el color de fondo del campo de texto, configura la propiedad `colors`. En lugar de un parámetro separado para cada color, el elemento componible contiene un parámetro combinado. Aquí, pasas una copia de la clase de datos `TextFieldDefaults`, en la que solo actualizas los colores que son diferentes. En este caso, se trata solo de los colores `unfocusedContainerColor` y `focusedContainerColor`.

En este paso, observaste el modo en que puedes usar los modificadores y los parámetros componibles para cambiar su apariencia y estilo. Esto se aplica a los elementos componibles que proporcionan las bibliotecas de Compose y Material, y a los que escribes por tu cuenta. Siempre debes pensar en brindar parámetros para personalizar el elemento que escribes. También debes agregar una propiedad `modifier` de modo que la apariencia del elemento se pueda adaptar desde el exterior.

[5. Align your body: Alineación](#)

El siguiente elemento componible que implementarás es el elemento "Align your body". Veamos su diseño, incluido el diseño de revisión que se muestra a su lado:

AlignYourBod...



El diseño de revisión ahora también contiene espaciados orientados a la línea base. Esta es la información que obtenemos de él:

- La imagen debe tener 88 dp de altura.
- El espaciado entre la línea base del texto y la imagen debe ser de 24 dp.

- El espaciado entre la línea base y la parte inferior del elemento debe ser de 8 dp.
- El texto debe tener un estilo tipográfico bodyMedium.

La **línea base** de un texto es la última línea en la que aparecen letras. Se les recomienda a los diseñadores alinear los elementos de texto en función de su línea base, en lugar de hacerlo en función de la parte superior o inferior.

Para implementar este elemento componible, necesitas los elementos [Image](#) y [Text](#) del mismo tipo. Deben incluirse en un [Column](#), por lo que se posicionan debajo de los demás.

Busca el elemento AlignYourBodyElement componible en tu código y actualiza su contenido con esta implementación básica:

```
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Column
import androidx.compose.ui.res.painterResource

@Composable
fun AlignYourBodyElement(
    modifier: Modifier = Modifier
){
    Column(
        modifier = modifier
    ){
        Image(
            painter = painterResource(R.drawable.ab1_inversions),
            contentDescription = null
        )
        Text(text = stringResource(R.string.ab1_inversions))
    }
}
```

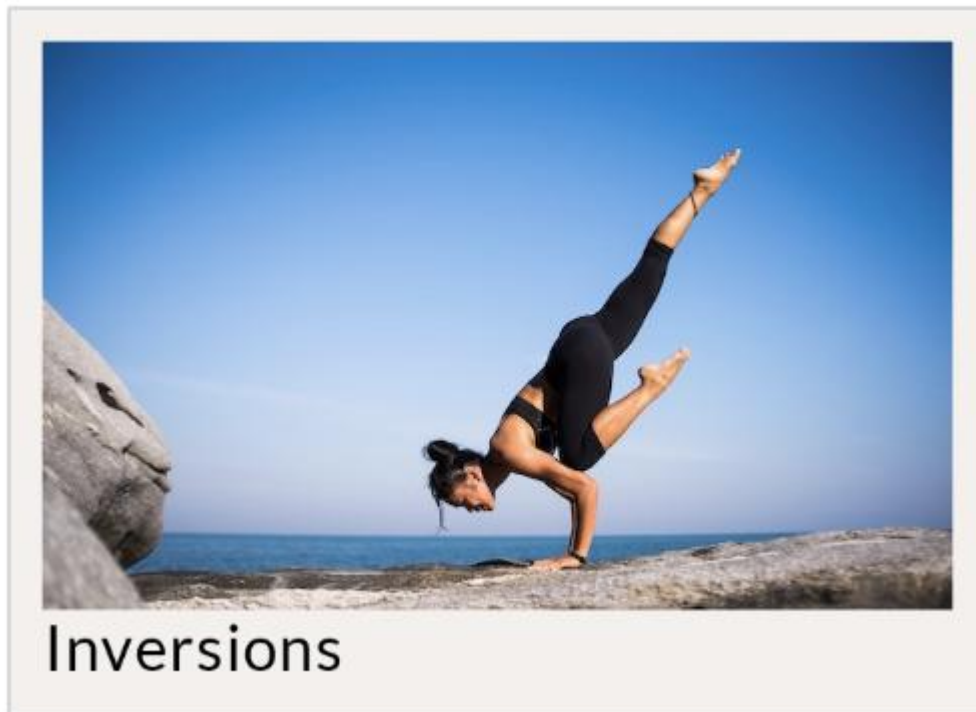
Observa lo siguiente:

- Debes establecer el contentDescription de la imagen como nulo, ya que esta es meramente decorativa. El texto debajo de la imagen es suficiente para describir el significado, por lo que no necesita una descripción adicional.

- Estás usando imagen y texto hard-coded. En el siguiente paso, los moverás para usar los parámetros proporcionados en el elemento `AlignYourBodyElement` componible y hacerlos dinámicos.

Observa la vista previa de este elemento componible:

AlignYourBodyElementPreview



Se deben realizar algunas mejoras. Lo más notable es que la imagen es demasiado grande y no tiene forma de círculo. Puedes adaptar el elemento `Image` componible con los modificadores [size](#) y [clip](#), y el parámetro `contentScale`.

El modificador `size` adapta el elemento que admite composición de modo que se ajuste a un tamaño determinado, similar a los modificadores `fillMaxWidth` y `heightIn` que viste en el paso anterior. El modificador `clip` funciona de manera diferente y **adapta la apariencia del elemento componible**. Puedes establecerla en cualquier [Shape](#) y recortará el contenido del elemento a esa forma.

```
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.ui.draw.clip
@Composable
```

```
fun AlignYourBodyElement(
    modifier: Modifier = Modifier
){
    Column(
        modifier = modifier
    ){
        Image(
            painter = painterResource(R.drawable.ab1_inversions),
            contentDescription = null,
            modifier = Modifier
                .size(88.dp)
                .clip(CircleShape)
        )
        Text(text = stringResource(R.string.ab1_inversions))
    }
}
```

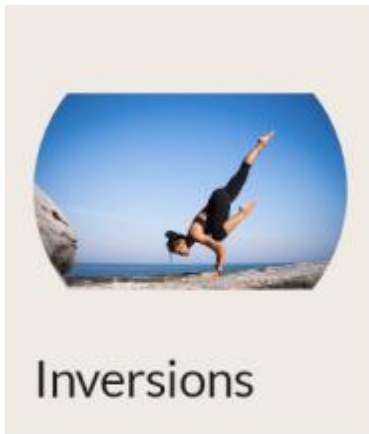
Actualmente, tu diseño en la vista previa se ve de la siguiente manera:

AlignYourBodyElementPre...



Inversions

La imagen también debe ajustarse correctamente. Para hacerlo, podemos usar el parámetro `contentScale` de `Image`. Existen varias opciones, en particular las siguientes:



Fit



FillBounds



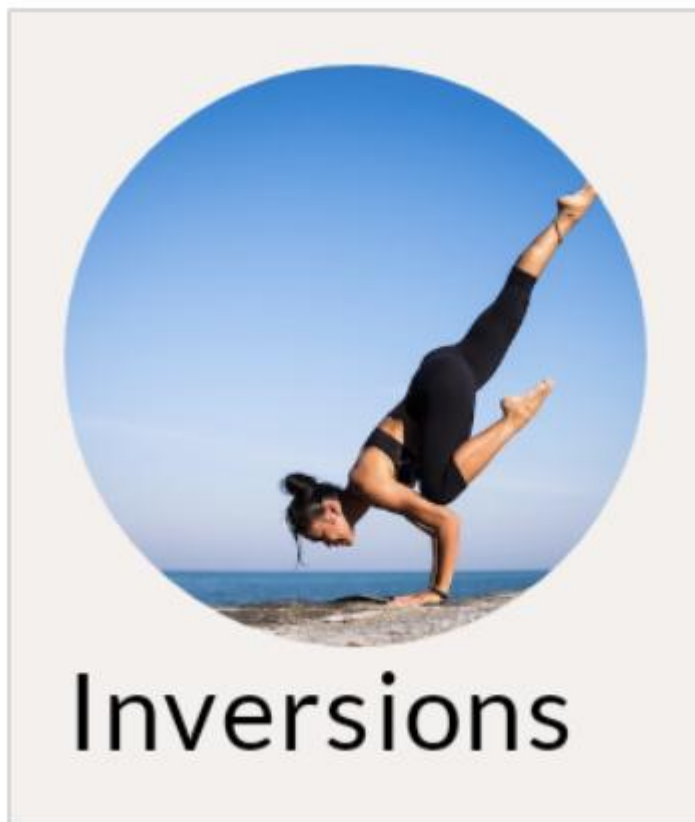
Crop

En este caso, el tipo de recorte es el correcto para usar. Después de aplicar los modificadores y el parámetro, tu código debería verse de la siguiente manera:

```
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.ui.draw.clip
import androidx.compose.ui.layout.ContentScale
@Composable
fun AlignYourBodyElement(
    modifier: Modifier = Modifier
){
    Column(
        modifier = modifier
    ){
        Image(
            painter = painterResource(R.drawable.ab1_inversions),
            contentDescription = null,
            contentScale = ContentScale.Crop,
            modifier = Modifier
                .size(88.dp)
                .clip(CircleShape)
        )
        Text( text = stringResource(R.string.ab1_inversions) )
    }
}
```

Tu diseño debería verse de la siguiente manera:

AlignYourBodyElementPre...



Inversions

Como siguiente paso, alinea el texto horizontalmente estableciendo la alineación de la Column.

En general, para alinear los elementos componibles dentro de un contenedor superior, debes establecer la **alineación** de ese contenedor superior. Por lo tanto, en lugar de indicarle al elemento secundario que se posicione en su elemento superior, debes indicarle al elemento superior el modo en que debe alinear sus elementos secundarios.

En una Column, tú decides el modo en que se deben alinear sus elementos secundarios de forma horizontal. Las opciones son las siguientes:

- Start
- CenterHorizontally

- End

En una Row, establecerás la alineación vertical. Las opciones son similares a las de Column:

- Top
- CenterVertically
- Abajo

En el caso de un elemento Box, combinarás la alineación horizontal y la vertical. Las opciones son las siguientes:

- TopStart
- TopCenter
- TopEnd
- CenterStart
- Centro
- CenterEnd
- BottomStart
- BottomCenter
- BottomEnd

Todos los elementos secundarios del contenedor seguirán este mismo patrón de alineación. Puedes anular el comportamiento de un solo elemento secundario si le agregas un modificador [align](#).

A los efectos de este diseño, el texto debe estar centrado horizontalmente. Para ello, configura el elemento `horizontalAlignment` de `Column` para que se centre de forma horizontal:

```
import androidx.compose.ui.Alignment
@Composable
fun AlignYourBodyElement(
    modifier: Modifier = Modifier
){
    Column(
        horizontalAlignment = Alignment.CenterHorizontally,
```

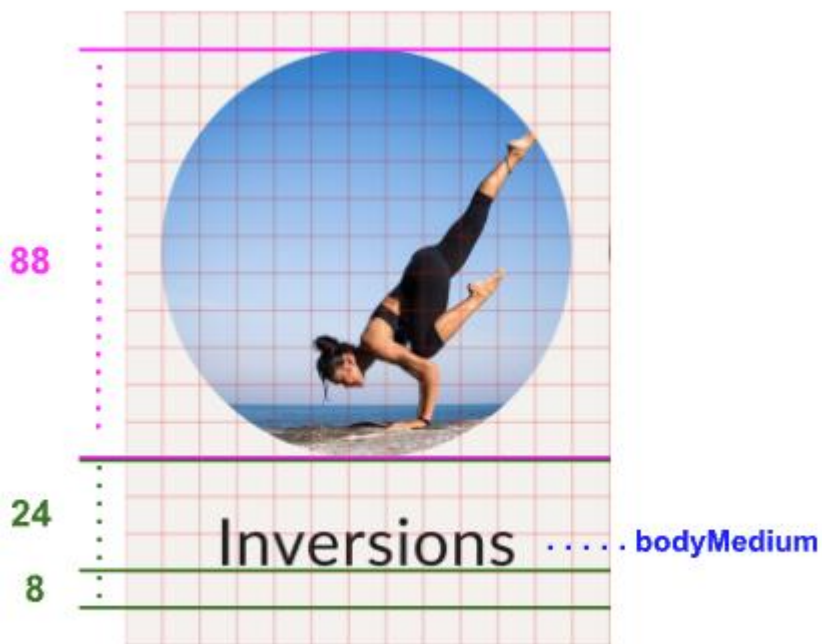
```

        modifier = modifier
    ){
        Image(
            //..
        )
        Text(
            //..
        )
    }
}

```

Con estas partes implementadas, solo hay algunos cambios menores que debes hacer para que el elemento componible sea idéntico al diseño. Intenta implementarlos por tu cuenta o consulta el código final si tienes algún problema. Piensa en los siguientes pasos:

- Haz que la imagen y el texto sean dinámicos. Pásalos como argumentos a la función de componibilidad. No olvides actualizar la vista previa correspondiente y pasar algunos datos hard-coded.
- Actualiza el texto de modo que use el estilo tipográfico `bodyMedium`.
- Actualiza el espaciado de referencia del elemento de texto según el diagrama.



Cuando termines de implementar estos pasos, tu código debería ser similar al siguiente:

```
import androidx.compose.foundation.layout.paddingFromBaseline
import androidx.compose.ui.Alignment
import androidx.compose.ui.layout.ContentScale

@Composable
fun AlignYourBodyElement(
    @DrawableRes drawable: Int,
    @StringRes text: Int,
    modifier: Modifier = Modifier
){
    Column(
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ){
        Image(
            painter = painterResource(drawable),
            contentDescription = null,
            contentScale = ContentScale.Crop,
            modifier = Modifier
                .size(88.dp)
                .clip(CircleShape)
        )
        Text(
            text = stringResource(text),
            modifier = Modifier.paddingFromBaseline(top = 24.dp, bottom = 8.dp),
            style = MaterialTheme.typography.bodyMedium
        )
    }
}

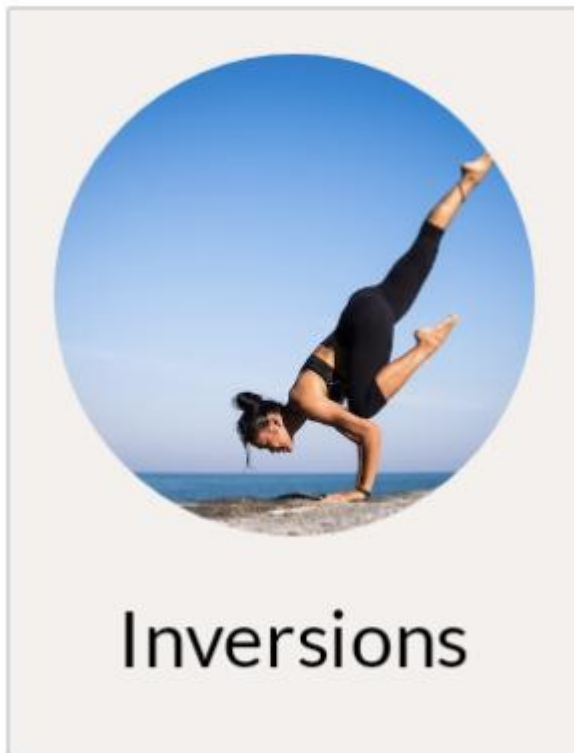
@Preview(showBackground = true, backgroundColor = 0xFFFF5F0EE)
@Composable
fun AlignYourBodyElementPreview() {
    MySootheTheme {
```

```
AlignYourBodyElement(
    text = R.string.ab1_inversions,
    drawable = R.drawable.ab1_inversions,
    modifier = Modifier.padding(8.dp)
)
}
}
```

A menudo, existen varias formas de obtener el mismo resultado, por lo que tu implementación puede diferir un poco de la solución propuesta. Por ejemplo, en general, puedes usar un [Spacer](#) o configurar un padding, y ambos tendrán el mismo resultado visual. Lo más importante es que tu implementación siga los lineamientos de diseño y Compose.

Consulta `AlignYourBodyElement` en la pestaña Design.

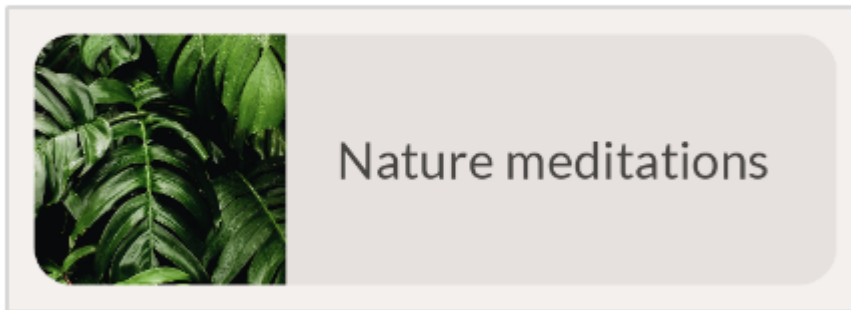
AlignYourBodyEleme...



[6. Tarjeta de Favorite collection: Elemento Surface de Material](#)

El siguiente elemento componible que implementaremos es similar al elemento "Align the body". Este es el diseño, incluida la revisión:

FavoriteCollectionCardPreview



En este caso, se proporciona el tamaño completo del elemento componible. Puedes ver que el texto debe ser `titleMedium`.

Este contenedor usa `surfaceVariant` como su color de fondo, que es diferente del fondo de toda la pantalla. También tiene esquinas redondeadas. Especificamos estos elementos para la tarjeta de "Favorite collection" con el elemento [Surface](#) componible de Material.

Surface es un componente de la biblioteca de Compose Material. Sigue los patrones generales de Material Design, y puedes adaptarlo cambiando el tema de tu app. Puedes obtener más información sobre los temas en el [codelab para crear temas en Compose](#) o en la [documentación sobre temas](#).

Adapta el componente `Surface` a tus necesidades configurando sus parámetros y modificadores. En este caso, la superficie debe tener esquinas redondeadas. Puedes usar el parámetro `shape` para esto. En lugar de establecer la forma en un

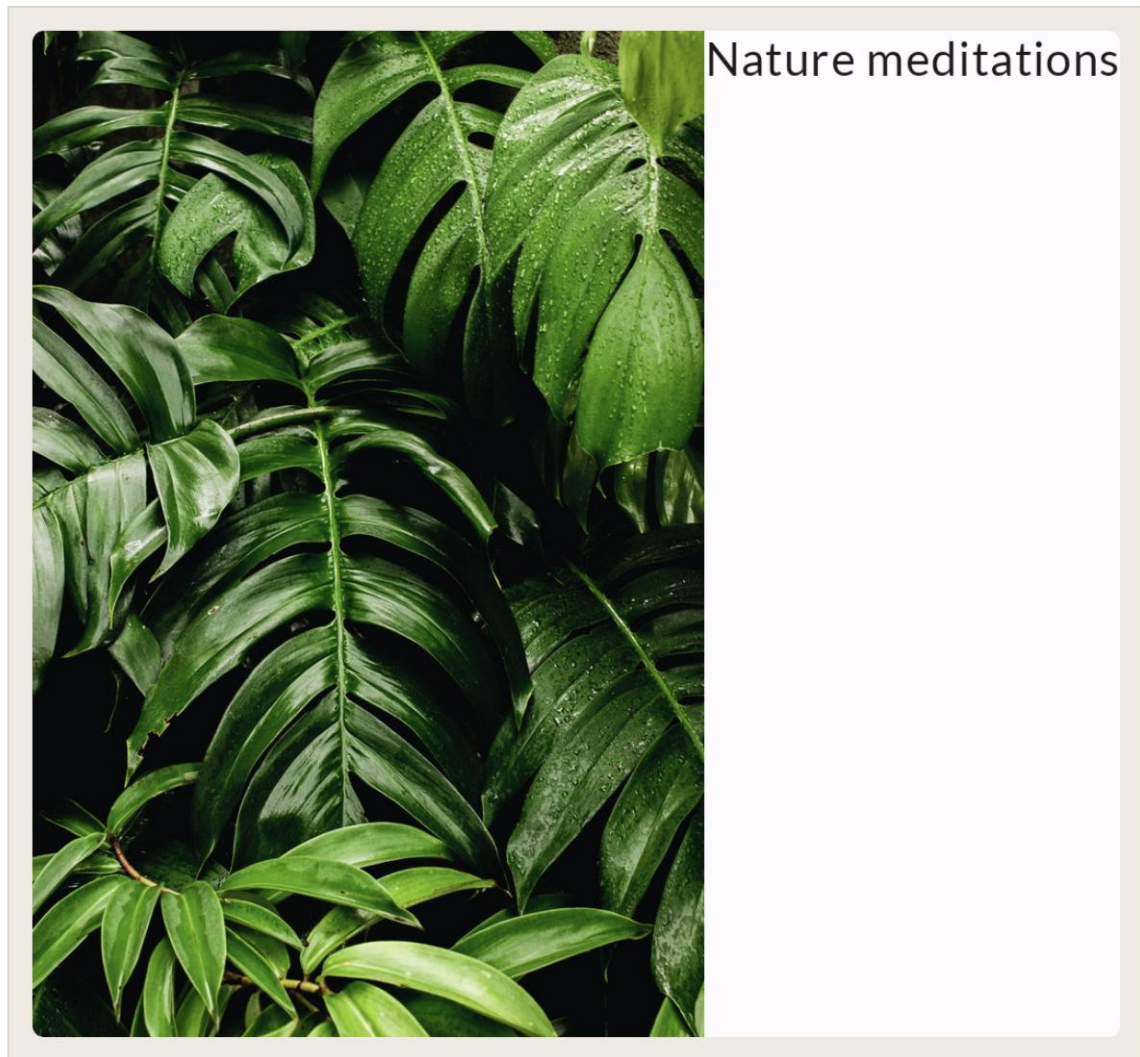
elemento Shape como en el caso de la imagen del paso anterior, usarás un valor que proviene de nuestro tema Material.

Veamos cómo se vería esto:

```
import androidx.compose.foundation.layout.Row
import androidx.compose.material3.Surface

@Composable
fun FavoriteCollectionCard(
    modifier: Modifier = Modifier
){
    Surface(
        shape = MaterialTheme.shapes.medium,
        modifier = modifier
    ){
        Row {
            Image(
                painter = painterResource(R.drawable.fc2_nature_meditations),
                contentDescription = null
            )
            Text(text = stringResource(R.string.fc2_nature_meditations))
        }
    }
}
```

Veamos la vista previa de esta implementación:



A continuación, aplica las lecciones aprendidas en el paso anterior.

- Configura el ancho de Row y alinea sus elementos secundarios de forma vertical.
- Establece el tamaño de la imagen según el diagrama y recórtala en su contenedor.



Intenta implementar estos cambios antes de ver el código de la solución.

El código debería verse de la siguiente manera:

```
import androidx.compose.foundation.layout.width
```

```
@Composable
```

```
fun FavoriteCollectionCard(
```

```
    modifier: Modifier = Modifier
```

```
) {
```

```
    Surface(
```

```
        shape = MaterialTheme.shapes.medium,
```

```
        modifier = modifier
```

```
    ) {
```

```
        Row(
```

```
            verticalAlignment = Alignment.CenterVertically,
```

```
            modifier = Modifier.width(255.dp)
```

```
        ) {
```

```
            Image(
```

```
                painter = painterResource(R.drawable.fc2_nature_meditations),
```

```
                contentDescription = null,
```

```
                contentScale = ContentScale.Crop,
```

```
                modifier = Modifier.size(80.dp)
```

```
            )
```

```
            Text(
```

```
                text = stringResource(R.string.fc2_nature_meditations)
```

```
            )
```

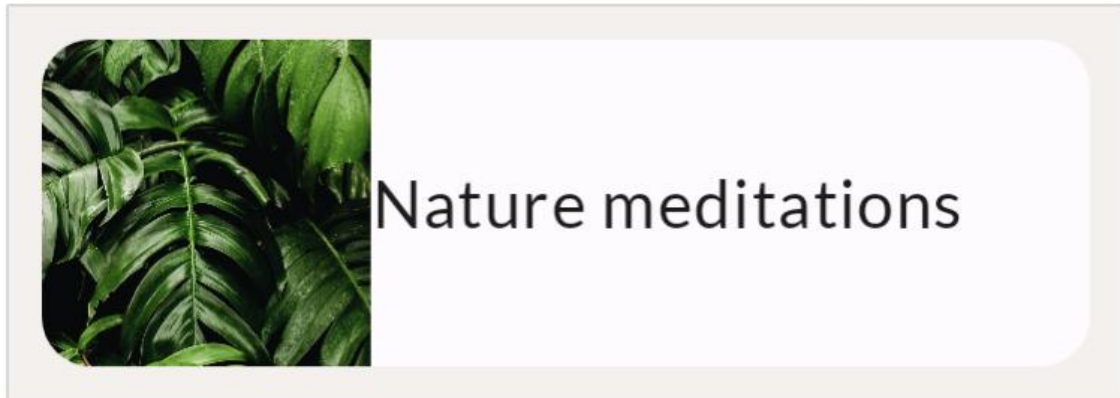
```
        }
```

```
    }
```

```
}
```

La vista previa debería tener el siguiente aspecto:

FavoriteCollectionCardPreview



Para finalizar este elemento componible, implementa los siguientes pasos:

- Haz que la imagen y el texto sean dinámicos. Pásalos como argumentos a la función de componibilidad.
- Actualiza el color a `surfaceVariant`.
- Actualiza el texto de modo que use el estilo tipográfico `titleMedium`.
- Actualiza el espaciado entre la imagen y el texto.

El resultado final debería ser similar al siguiente:

```
@Composable
fun FavoriteCollectionCard(
    @DrawableRes drawable: Int,
    @StringRes text: Int,
    modifier: Modifier = Modifier
){
    Surface(
        shape = MaterialTheme.shapes.medium,
        color = MaterialTheme.colorScheme.surfaceVariant,
        modifier = modifier
    ){
        Row(
```

```

        verticalAlignment = Alignment.CenterVertically,
        modifier = Modifier.width(255.dp)
    ){
        Image(
            painter = painterResource(drawable),
            contentDescription = null,
            contentScale = ContentScale.Crop,
            modifier = Modifier.size(80.dp)
        )
        Text(
            text = stringResource(text),
            style = MaterialTheme.typography.titleMedium,
            modifier = Modifier.padding(horizontal = 16.dp)
        )
    }
}
}

```

//..

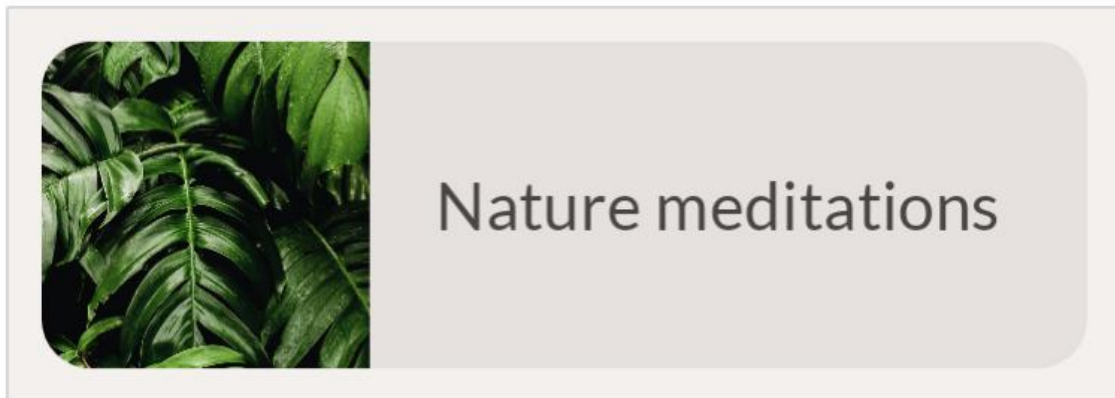
```

@Preview(showBackground = true, backgroundColor = 0xFFFF5F0EE)
@Composable
fun FavoriteCollectionCardPreview() {
    MySootheTheme {
        FavoriteCollectionCard(
            text = R.string.fc2_nature_meditations,
            drawable = R.drawable.fc2_nature_meditations,
            modifier = Modifier.padding(8.dp)
        )
    }
}

```

Consulta la vista previa de FavoriteCollectionCardPreview.

FavoriteCollectionCardPreview



7. Fila de Align your body: Disposiciones

Ahora que creaste los elementos básicos componibles que se muestran en la pantalla, puedes comenzar a crear sus diferentes secciones en ella.

Comienza con la fila desplazable de "Align your body".



A continuación, se muestra el diseño de revisión para este componente:



Recuerda que un bloque de la cuadrícula representa 8 dp. En este diseño, hay 16 dp de espacio antes del primer elemento de la fila y después del último. Hay 8 dp de espaciado entre cada elemento.

En Compose, puedes implementar una fila desplazable como esta con el elemento [LazyRow](#) componible. La [documentación sobre listas](#) contiene mucha más información sobre las listas diferidas, como LazyRow y LazyColumn. En este codelab, basta con saber que LazyRow solo renderiza los elementos que se muestran en pantalla en lugar de todos los elementos al mismo tiempo, lo que ayuda a mantener el rendimiento de la app.

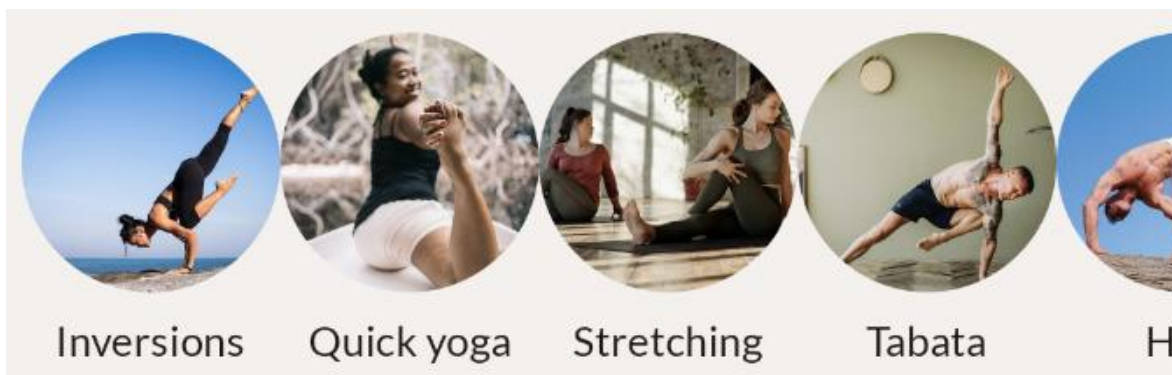
Comienza con una implementación básica de este LazyRow:

```
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
```

```
@Composable
fun AlignYourBodyRow(
    modifier: Modifier = Modifier
){
    LazyRow(
        modifier = modifier
    ){
        items(alignYourBodyData) { item ->
            AlignYourBodyElement(item.drawable, item.text)
        }
    }
}
```

Como puedes ver, los elementos secundarios de un LazyRow no son elementos componibles. En cambio, debes usar el DSL de lista diferida que proporciona métodos como [item](#) y [items](#) que emiten elementos componibles como los de lista. Para cada elemento en el componente alignYourBodyData proporcionado, emites un elemento AlignYourBodyElement componible que implementaste antes.

Observa cómo se muestra:

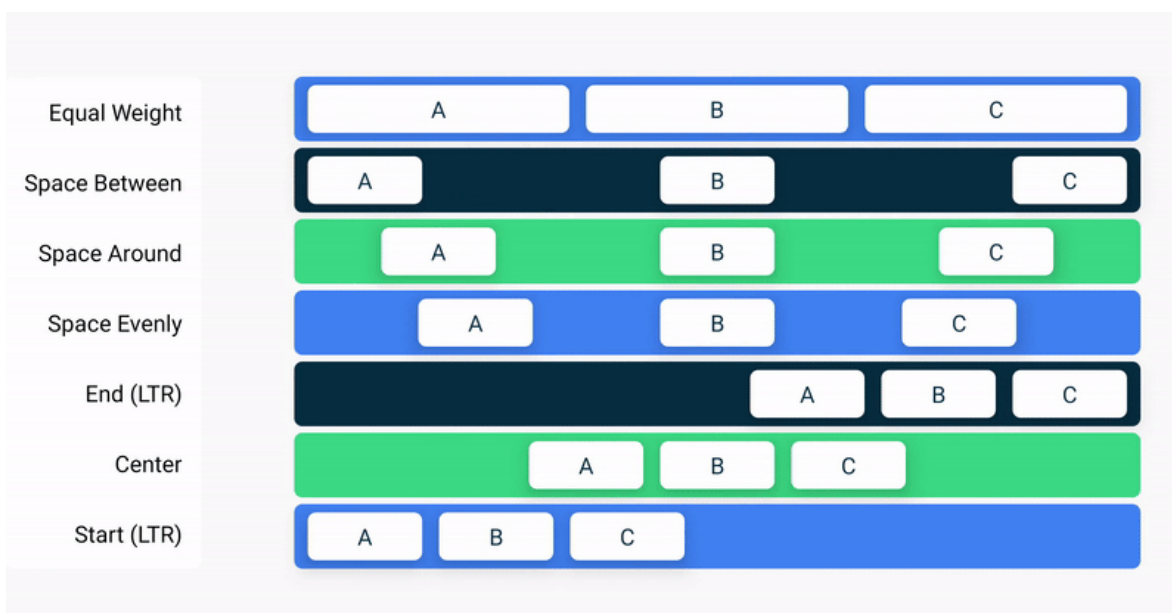


Todavía falta el espaciado que vimos en el diseño de revisión. Para implementarlo, deberás aprender sobre **disposiciones**.

En el paso anterior, conociste las alineaciones, que se usan para alinear los elementos secundarios de un contenedor en el **eje cruzado**. En el caso de una Column, el eje cruzado es el horizontal, mientras que, en el caso de una Row, el eje cruzado es el vertical.

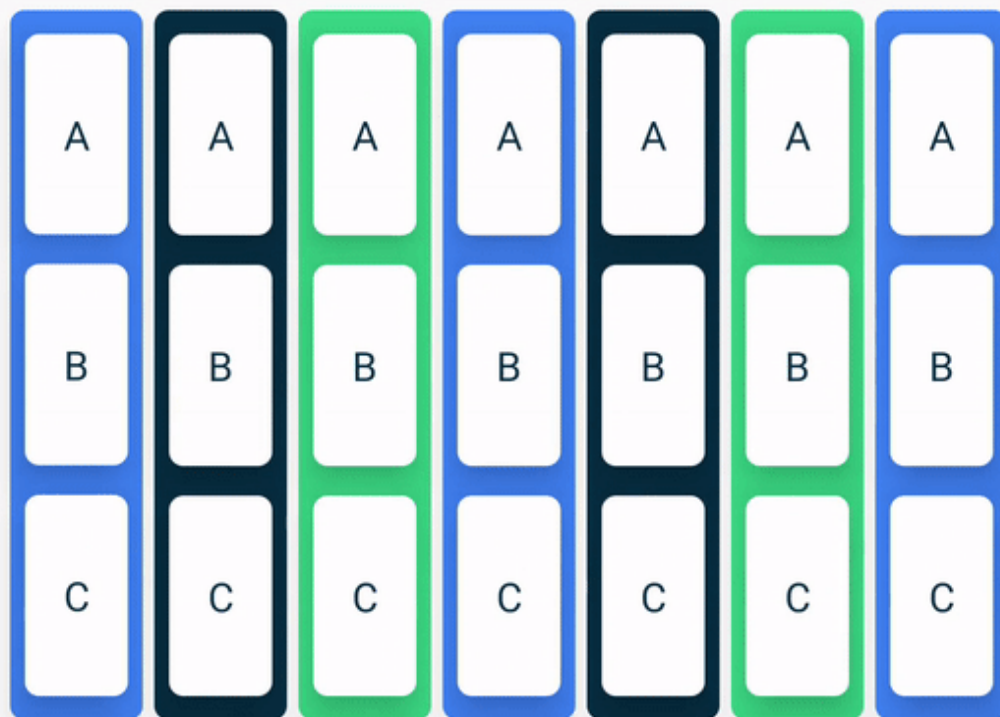
Sin embargo, también podemos tomar una decisión sobre la forma de ubicar los elementos secundarios que admiten composición en el **eje principal** de un contenedor (el horizontal para una Row, el vertical para una Column).

En el caso de una Row, puedes elegir las siguientes disposiciones:



Y para una Column:

Equal Weight Space Between Space Around Space Evenly Top Center Bottom



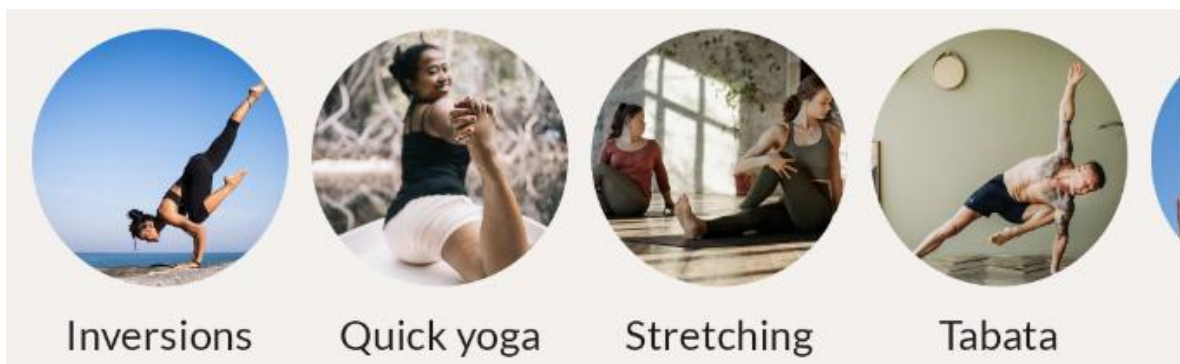
Además de estas disposiciones, también puedes usar el método `Arrangement.spacedBy()` a fin de agregar un espacio fijo entre cada elemento secundario que admite composición.

En el ejemplo, el método `spacedBy` es el que debes usar, ya que quieres colocar 8 dp de espacio entre cada elemento en `LazyRow`.

```
import androidx.compose.foundation.layout.Arrangement
```

```
@Composable
fun AlignYourBodyRow(
    modifier: Modifier = Modifier
){
    LazyRow(
        horizontalArrangement = Arrangement.spacedBy(8.dp),
        modifier = modifier
    ){
        items(alignYourBodyData) { item ->
            AlignYourBodyElement(item.drawable, item.text)
        }
    }
}
```

Ahora, el diseño se ve así:



También debes agregar padding a los costados del elemento `LazyRow`. En este caso, no es útil agregar un modificador de padding simple. Intenta agregar padding a `LazyRow` y observa su comportamiento con la [vista previa interactiva](#):



Como puedes ver, cuando te desplazas, el primer y el último elemento visible están cortados en ambos lados de la pantalla.

Para mantener el mismo padding, pero también desplazar tu contenido dentro de los límites de la lista superior sin recortarlo, todas las listas proporcionan un parámetro a LazyRow llamado `contentPadding` y lo establece en 16.dp.

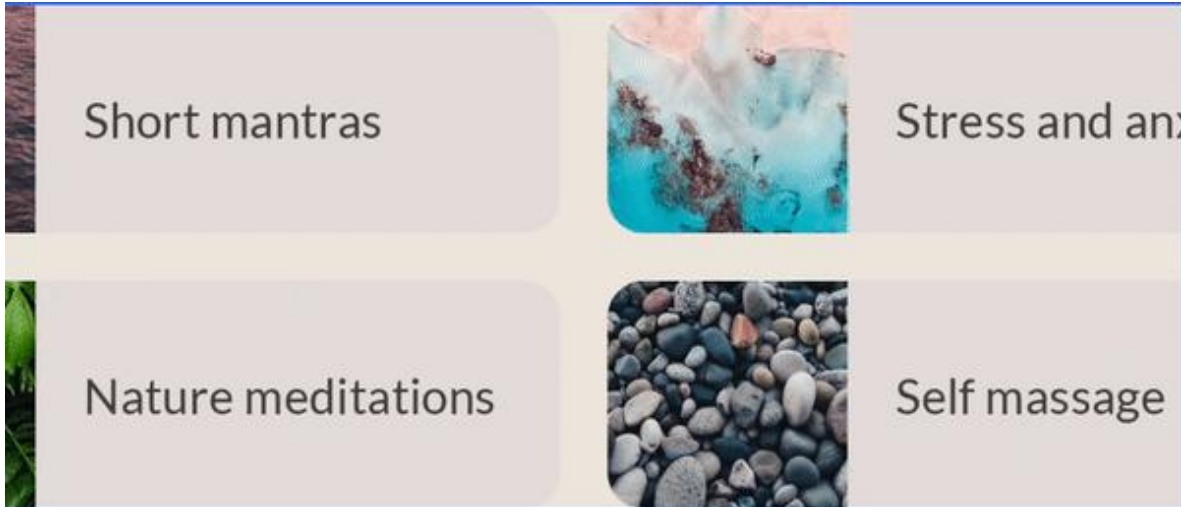
Asegúrate de que el elemento componible se comporte de manera correcta cuando deslizas la lista. Puedes usar la [vista previa interactiva](#) para interactuar con la vista previa del elemento componible.

```
import androidx.compose.foundation.layout.PaddingValues
```

```
@Composable
fun AlignYourBodyRow(
    modifier: Modifier = Modifier
){
    LazyRow(
        horizontalArrangement = Arrangement.spacedBy(8.dp),
        contentPadding = PaddingValues(horizontal = 16.dp),
        modifier = modifier
    ){
        items(alignYourBodyData) { item ->
            AlignYourBodyElement(item.drawable, item.text)
        }
    }
}
```

8. Cuadrícula de Favorite collections: Cuadrículas diferidas

La siguiente sección para implementar es la sección "Favorite collections" de la pantalla. En lugar de una sola fila, este elemento necesita una cuadrícula:



Podrías implementar esta sección de manera similar a la anterior si creas una LazyRow y permites que cada elemento contenga una Column con dos instancias de FavoriteCollectionCard. Sin embargo, en este paso usarás la [LazyHorizontalGrid](#), que proporciona una asignación más atractiva de elementos a elementos de cuadrícula.

Comienza con una implementación simple de la cuadrícula con dos filas fijas:

```
import androidx.compose.foundation.lazy.grid.GridCells
import androidx.compose.foundation.lazy.grid.LazyHorizontalGrid
import androidx.compose.foundation.lazy.grid.items
```

```
@Composable
fun FavoriteCollectionsGrid(
    modifier: Modifier = Modifier
){
    LazyHorizontalGrid(
        rows = GridCells.Fixed(2),
        modifier = modifier
    ){
        items(favoriteCollectionsData) { item ->
            FavoriteCollectionCard(item.drawable, item.text)
        }
    }
}
```

```
}  
}
```

Como puedes ver, simplemente reemplazaste el LazyRow del paso anterior por una LazyHorizontalGrid. Sin embargo, esto todavía no te dará el resultado correcto:

FavoriteCollectionsGridPreview



Short mantras



Stress



Nature meditations



Self m

La cuadrícula ocupa tanto espacio como su elemento superior, lo que significa que las tarjetas de colecciones favoritas están demasiado estiradas verticalmente.

Adapta el elemento componible de modo que se cumpla lo siguiente:

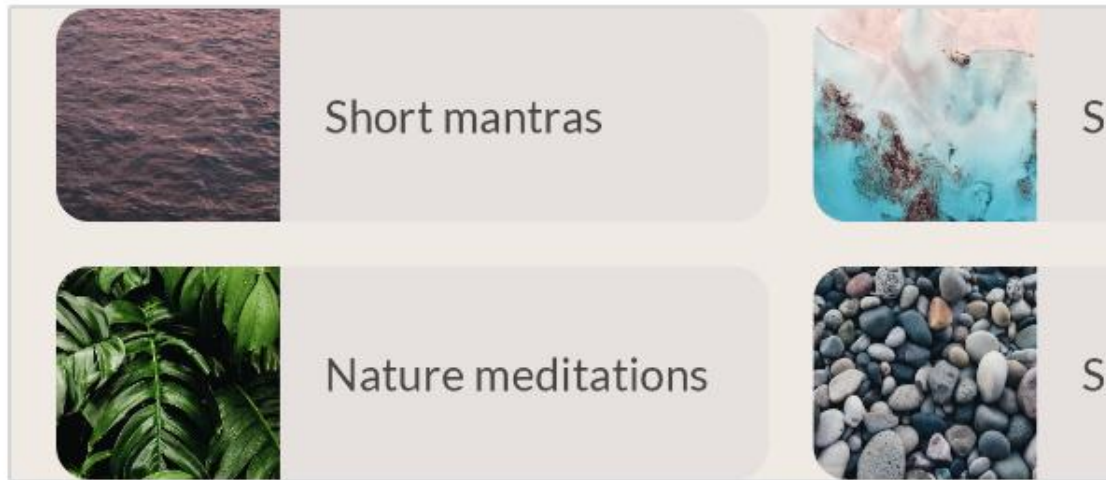
- La cuadrícula tiene `contentPadding` horizontal de 16 dp.
- La disposición horizontal y vertical tiene espacios de 16 dp.
- La altura de la cuadrícula es de 168 dp.
- El modificador de `FavoriteCollectionCard` especifica una altura de 80 dp.

El código final debería verse así:

```
@Composable
fun FavoriteCollectionsGrid(
    modifier: Modifier = Modifier
){
    LazyHorizontalGrid(
        rows = GridCells.Fixed(2),
        contentPadding = PaddingValues(horizontal = 16.dp),
        horizontalArrangement = Arrangement.spacedBy(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp),
        modifier = modifier.height(168.dp)
    ){
        items(favoriteCollectionsData) { item ->
            FavoriteCollectionCard(item.drawable, item.text, Modifier.height(80.dp))
        }
    }
}
```

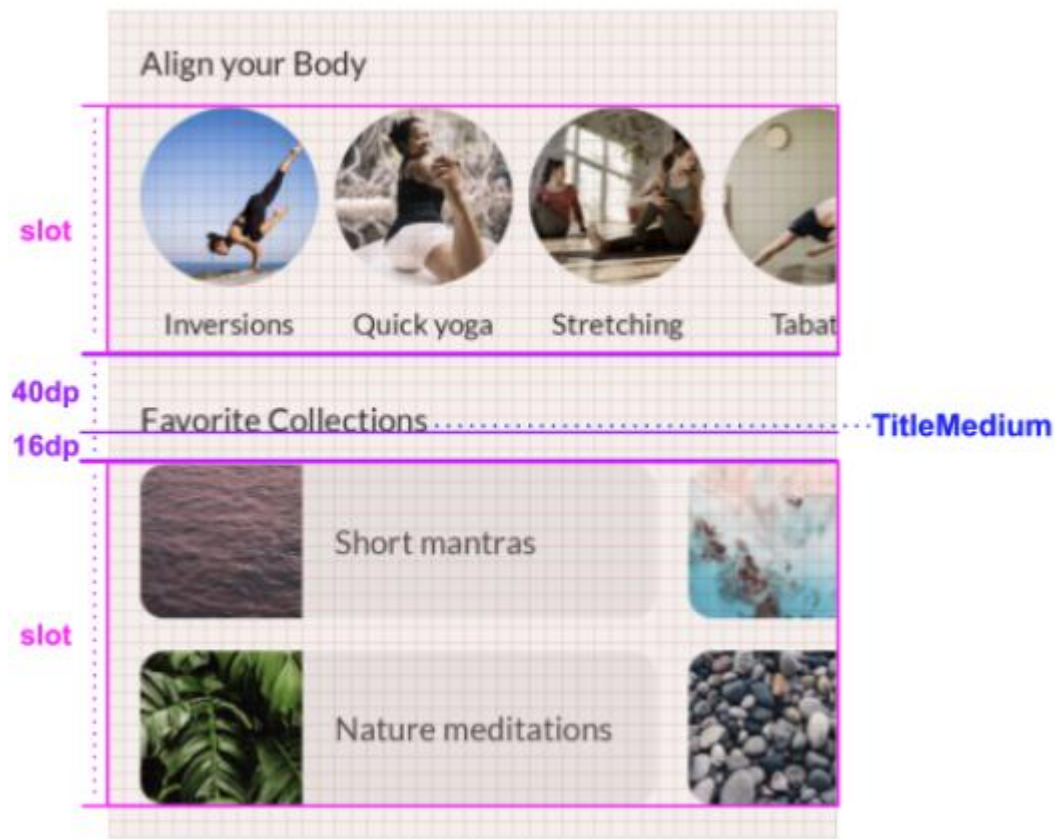
La vista previa debería verse de la siguiente manera:

FavoriteCollectionsGridPreview



[9. Sección principal: APIs de ranuras](#)

En la pantalla principal de MySoothe, hay varias **secciones** que siguen el mismo patrón. Cada una tiene un título, y parte del contenido varía según la sección. Este es el diseño que queremos implementar:



Como puedes ver, cada sección tiene un **título** y una **ranura**. El título tiene asociada determinada información como el estilo y el espaciado. La ranura se puede rellenar de forma dinámica con contenido diferente, según la sección.

Para implementar este contenedor flexible de secciones, usa las llamadas *APIs de ranuras*. Antes de implementar esto, lee la sección sobre [diseños basados en ranuras](#) en la documentación. Esto te ayudará a comprender lo que es un diseño basado en ranuras y el modo en que puedes usar las APIs de ranuras a fin de compilar diseños de este tipo.

Los **diseños basados en ranuras** dejan un espacio vacío en la IU para que el desarrollador lo complete como quiera. Puedes usarlos para crear diseños más flexibles.

Adapta el elemento componible HomeSection de modo que reciba el título y el contenido de la ranura. También debes adaptar la vista previa asociada para llamar a esta HomeSection con el título y el contenido de "Align your body":

```
@Composable
fun HomeSection(
```

```

@StringRes title: Int,
modifier: Modifier = Modifier,
content: @Composable () -> Unit
){
    Column(modifier) {
        Text(stringResource(title))
        content()
    }
}

@Preview(showBackground = true, backgroundColor = 0xFFFF5F0EE)
@Composable
fun HomeSectionPreview() {
    MySootheTheme {
        HomeSection(R.string.align_your_body) {
            AlignYourBodyRow()
        }
    }
}

```

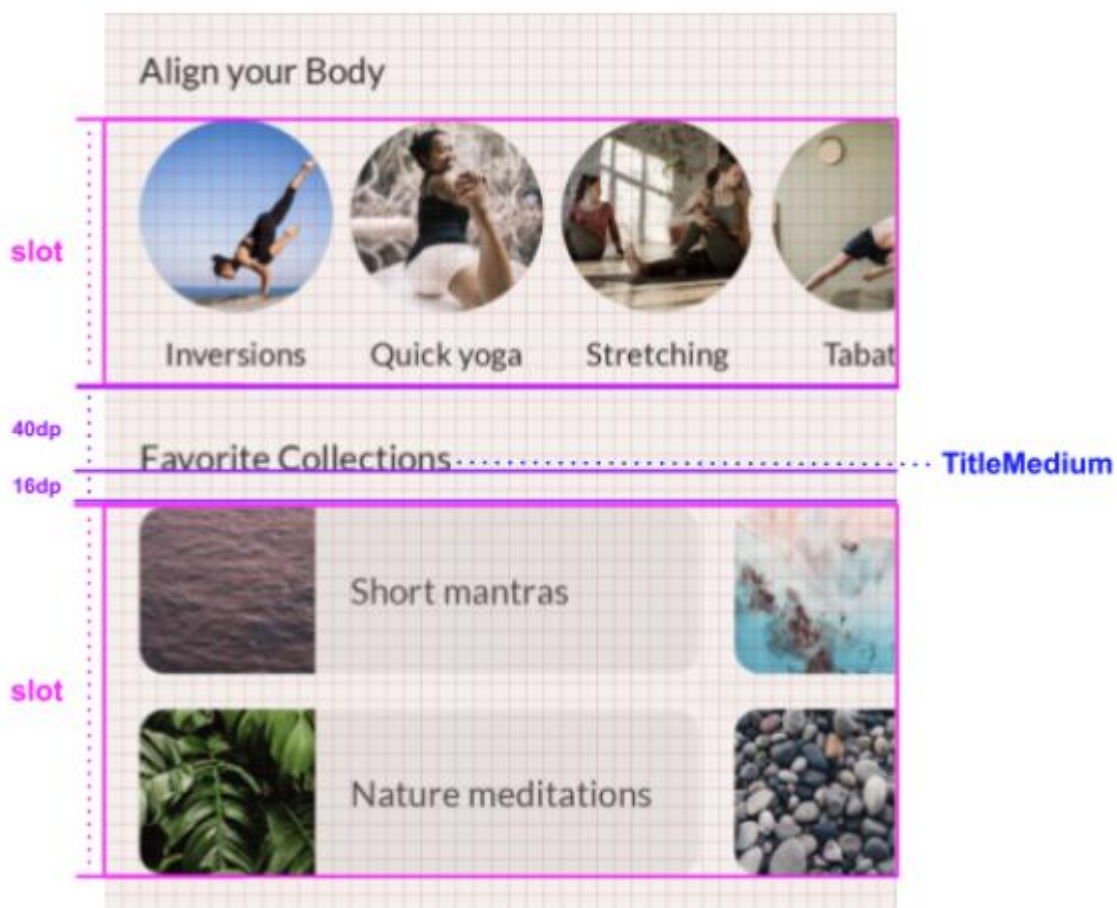
Puedes usar el parámetro `content` para la ranura del elemento componible. De esta manera, cuando usas el elemento `HomeSection`, puedes usar una expresión lambda final a los efectos de llenar la ranura de contenido. Cuando un elemento componible proporciona varias ranuras para completar, puedes asignarles nombres significativos que representen su función en el contenedor más grande. Por ejemplo, la [TopAppBar](#) de Material proporciona las ranuras para el `title`, el `navigationIcon` y las `actions`.

Veamos cómo se ve la sección con esta implementación:

HomeSectionPreview



El elemento de texto componible necesita más información para alinearse con el diseño.



Actualízalo de modo que el texto tenga las siguientes características:

- Usa la tipografía titleMedium.
- El espaciado entre la línea base del texto y la parte superior es de 40 dp.
- El espaciado entre la línea base y la parte inferior del elemento es de 16 dp.
- El padding horizontal es de 16 dp.

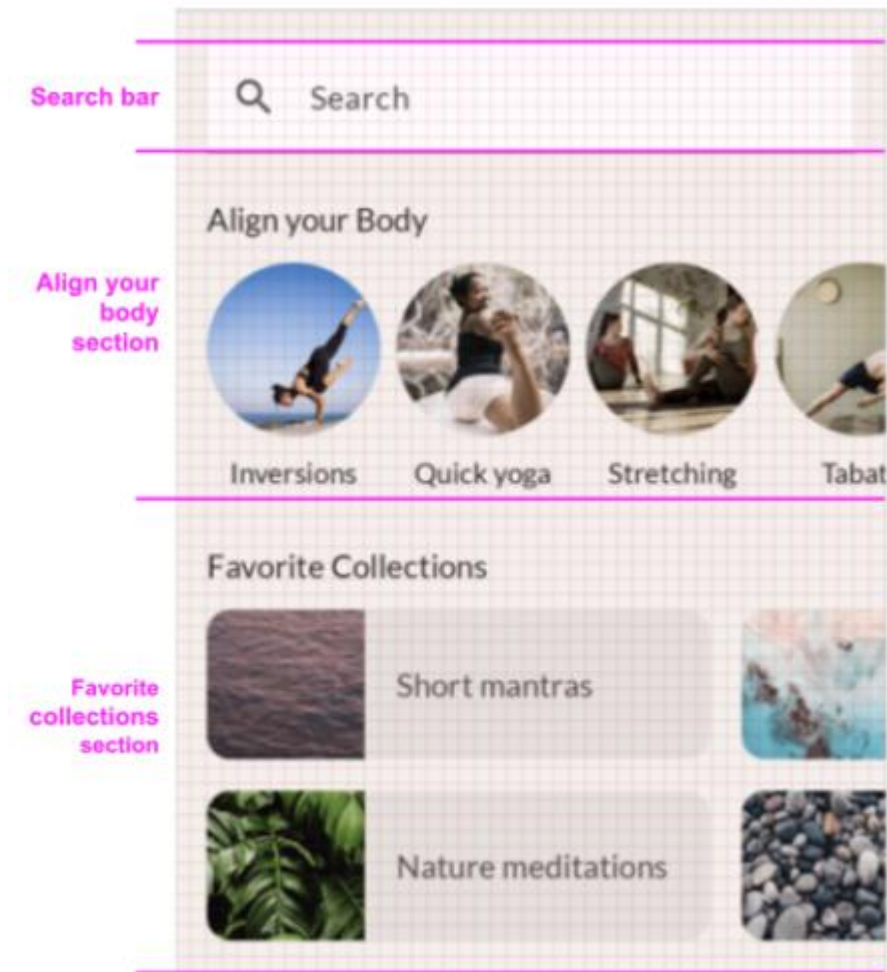
La solución final debería ser similar a la siguiente:

```
@Composable
fun HomeSection(
    @StringRes title: Int,
    modifier: Modifier = Modifier,
    content: @Composable () -> Unit
){
    Column(modifier) {
        Text(
            text = stringResource(title),
            style = MaterialTheme.typography.titleMedium,
            modifier = Modifier
                .paddingFromBaseline(top = 40.dp, bottom = 16.dp)
                .padding(horizontal = 16.dp)
        )
        content()
    }
}
```

[10. Pantalla principal: Desplazamiento](#)

Ahora que creaste todos los componentes fundamentales, puedes combinarlos en una implementación de pantalla completa.

Este es el diseño que intentas implementar:



Simplemente estamos ubicando la barra de búsqueda y las dos secciones, una debajo de la otra. Debes agregar un espaciado de modo que todo encaje en el diseño. Un elemento componible que no usamos antes es [Spacer](#), que nos ayuda a colocar espacio adicional dentro de nuestra Column. Si quieres, en cambio, establecer el padding de la Column, obtendrás el mismo comportamiento de corte que vimos antes en la cuadrícula de Favorite collections.

@Composable

```
fun HomeScreen(modifier: Modifier = Modifier) {
    Column(modifier) {
        Spacer(Modifier.height(16.dp))
        SearchBar(Modifier.padding(horizontal = 16.dp))
        HomeSection(title = R.string.align_your_body) {
            AlignYourBodyRow()
        }
    }
}
```

```

HomeSection(title = R.string.favorite_collections) {
    FavoriteCollectionsGrid()
}
Spacer(Modifier.height(16.dp))
}
}

```

Aunque el diseño se adapta bien a la mayoría de los tamaños de dispositivos, es necesario que se pueda desplazar de forma vertical en caso de que el dispositivo no sea lo suficientemente alto, por ejemplo, cuando está en modo horizontal. Esto requiere que agregues un comportamiento de desplazamiento.

Como vimos antes, los diseños diferidos como `LazyRow` y `LazyHorizontalGrid` agregan automáticamente el comportamiento de desplazamiento. Sin embargo, no siempre necesitas un diseño diferido. En general, **se usa un diseño diferido cuando se tienen muchos elementos en una lista o grandes conjuntos de datos para cargar**, por lo que emitir todos los elementos a la vez tendría un costo de rendimiento y ralentizaría tu app. Cuando una lista tiene solo una cantidad limitada de elementos, puedes optar por usar una `Column` o `Row` simple y **agregar el comportamiento de desplazamiento de forma manual**. A tal fin, usa los modificadores [verticalScroll](#) o [horizontalScroll](#). Estos requieren un elemento [ScrollState](#), que contiene el estado actual del desplazamiento y que se usa para modificar el estado del desplazamiento desde el exterior. En este caso, no quieres modificar el estado de desplazamiento, por lo que solo debes crear una instancia de `ScrollState` persistente usando [rememberScrollState](#).

Si deseas obtener más información sobre `remember` y su función en el estado de Compose, sigue el [codelab Estado en Compose](#).

El resultado final debería verse de la siguiente manera:

```

import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll

@Composable
fun HomeScreen(modifier: Modifier = Modifier) {
    Column(
        modifier
            .verticalScroll(rememberScrollState())
    )
}

```

```

){
    Spacer(Modifier.height(16.dp))
    SearchBar(Modifier.padding(horizontal = 16.dp))
    HomeSection(title = R.string.align_your_body) {
        AlignYourBodyRow()
    }
    HomeSection(title = R.string.favorite_collections) {
        FavoriteCollectionsGrid()
    }
    Spacer(Modifier.height(16.dp))
}
}
}

```

Si deseas verificar el comportamiento de desplazamiento del elemento componible, limita la altura de la vista previa y ejecútala en la [vista previa interactiva](#):

```

@Preview(showBackground = true, backgroundColor = 0xFFFF5F0EE, heightDp = 180)
@Composable
fun ScreenContentPreview() {
    MySootheTheme { HomeScreen() }
}

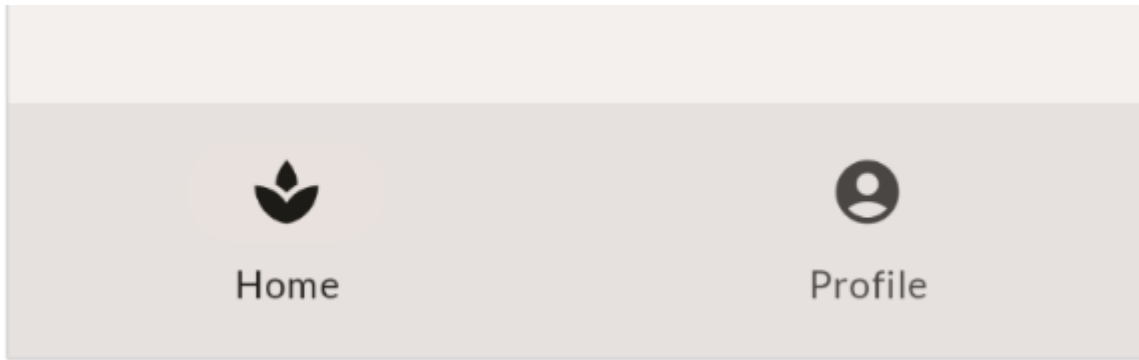
```

11. Navegación inferior: Material

Ahora que implementaste el contenido de la pantalla, podrás agregar la decoración de la ventana. En el caso de MySoothe, existe una barra de navegación que permite al usuario alternar entre diferentes pantallas.

Primero, implementa la barra de navegación componible y, luego, inclúyela en tu app.

Echemos un vistazo al diseño:



Por fortuna, no tienes que implementar este elemento componible desde cero. Puedes usar el elemento [NavigationBar](#) componible que forma parte de la biblioteca de Compose Material. Dentro del elemento `NavigationBar` componible, puedes agregar un elemento `NavigationBarItem` o más. Luego, la biblioteca de Material les dará estilo automáticamente.

Si te interesa **Material Design** y quieres obtener más información para implementar un sistema de diseño con Jetpack Compose, sigue el [codelab para crear temas en Compose](#) o lee la [documentación sobre temas](#).

Comienza con una implementación básica de esta navegación inferior:

```
import androidx.compose.material3.NavigationBar
import androidx.compose.material3.NavigationBarItem
import androidx.compose.material.icons.filled.AccountCircle
import androidx.compose.material.icons.filled.Spa

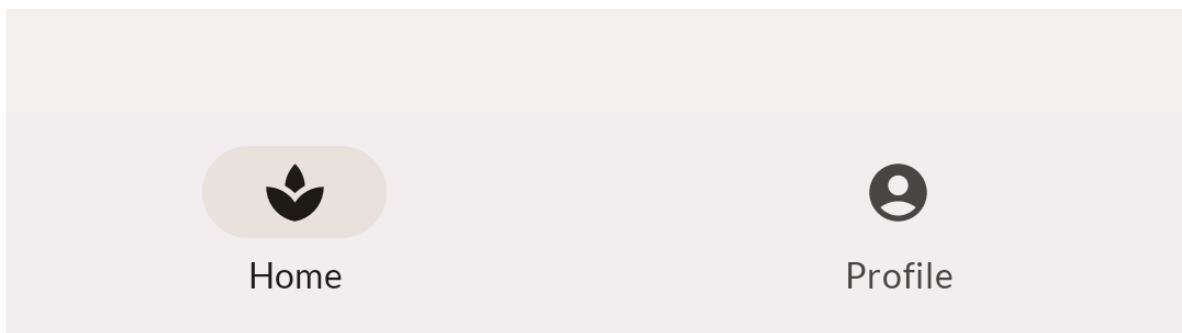
@Composable
private fun SootheBottomNavigation(modifier: Modifier = Modifier) {
    NavigationBar(
        modifier = modifier
    ){
        NavigationBarItem(
            icon = {
                Icon(
                    imageVector = Icons.Default.Spa,
                    contentDescription = null
                )
            },
            label = {
                Text(
```

```

        text = stringResource(R.string.bottom_navigation_home)
    )
},
selected = true,
onClick = {}
)
NavigationBarItem(
    icon = {
        Icon(
            imageVector = Icons.Default.AccountCircle,
            contentDescription = null
        )
    },
    label = {
        Text(
            text = stringResource(R.string.bottom_navigation_profile)
        )
    },
    selected = false,
    onClick = {}
)
}
}

```

Este es el aspecto de la implementación básica: no hay mucho contraste entre el color del contenido y el color de la barra de navegación.

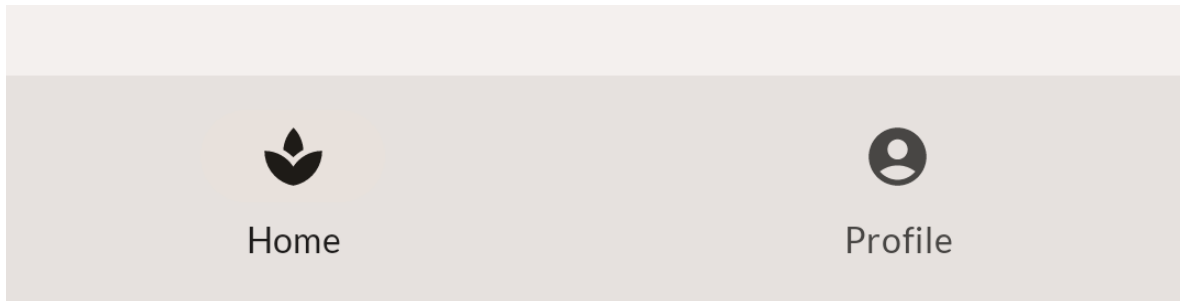


Hay algunas adaptaciones de estilo que deberías hacer. En primer lugar, puedes actualizar el color de fondo de la navegación inferior si configuras el

parámetro `containerColor`. Para ello, puedes usar el color `surfaceVariant` del tema de Material. La solución final debería ser similar a la siguiente:

```
@Composable
private fun SootheBottomNavigation(modifier: Modifier = Modifier) {
    NavigationBar(
        containerColor = MaterialTheme.colorScheme.surfaceVariant,
        modifier = modifier
    ){
        NavigationBarItem(
            icon = {
                Icon(
                    imageVector = Icons.Default.Spa,
                    contentDescription = null
                )
            },
            label = {
                Text(stringResource(R.string.bottom_navigation_home))
            },
            selected = true,
            onClick = {}
        )
        NavigationBarItem(
            icon = {
                Icon(
                    imageVector = Icons.Default.AccountCircle,
                    contentDescription = null
                )
            },
            label = {
                Text(stringResource(R.string.bottom_navigation_profile))
            },
            selected = false,
            onClick = {}
        )
    }
}
```

Ahora la barra de navegación debería verse así (observa que proporciona más contraste):



12. App de MySoothe: Scaffold

Para este paso, crea la implementación de pantalla completa, incluida la navegación inferior. Usa el elemento [Scaffold](#) componible de Material. Scaffold te proporciona un **elemento configurable componible de nivel superior** a los efectos de usarlo en apps que implementan Material Design. Contiene ranuras para varios conceptos de Material, uno de los cuales es la barra inferior. En esta barra, puedes colocar el elemento de navegación inferior componible que creaste en el paso anterior.

Implementa el elemento `MySootheAppPortrait()` componible. Este es el elemento de nivel superior que usarás en tu app, por lo que debes hacer lo siguiente:

- Aplica el tema `MySootheTheme` de Material.
- Agrega el componente `Scaffold`.
- Establece la barra inferior de modo que sea el elemento de la `SootheBottomNavigation` componible.
- Configura el contenido de modo que sea el elemento de la `HomeScreen` componible.

El resultado final debería ser el siguiente:

```
import androidx.compose.material3.Scaffold
```

```
@Composable
fun MySootheAppPortrait() {
    MySootheTheme {
        Scaffold(
```

```
        bottomBar = { SootheBottomNavigation() }  
    ){ padding ->  
        HomeScreen(Modifier.padding(padding))  
    }  
}  
}
```

La implementación ya está completa. Si quieres verificar si tu versión se implementa de manera perfecta para píxeles, puedes comparar esta imagen con tu propia implementación de vista previa.

 Search

Align your Body



Inversions



Quick yoga



Stretching



Tabat

Favorite Collections



Short mantras



Nature meditations



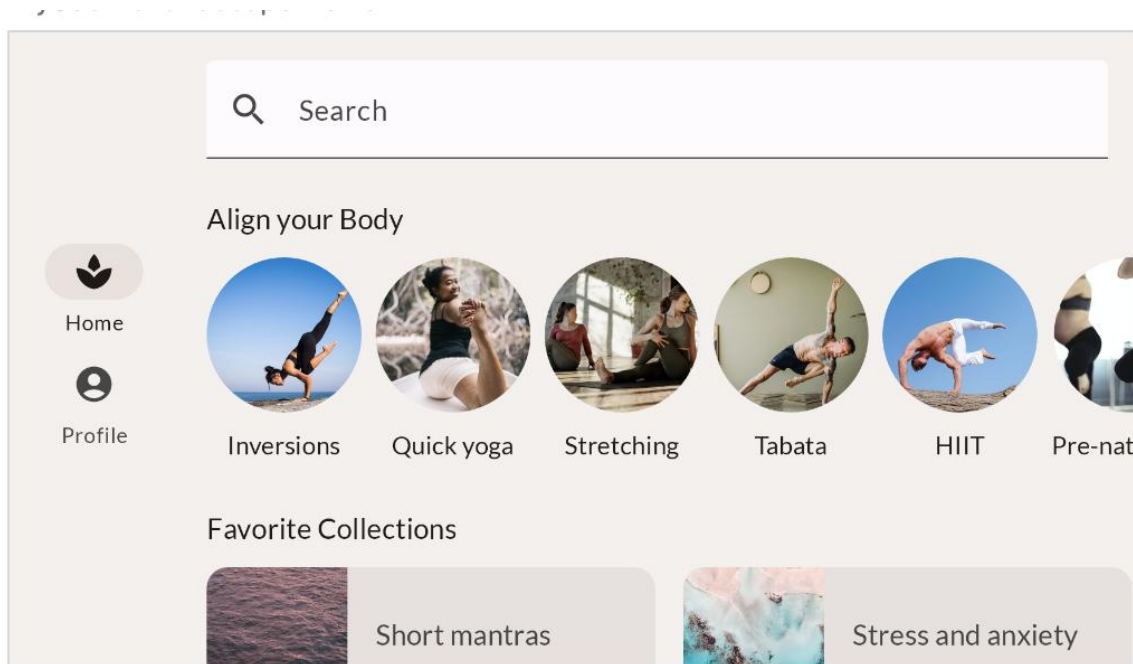
Home



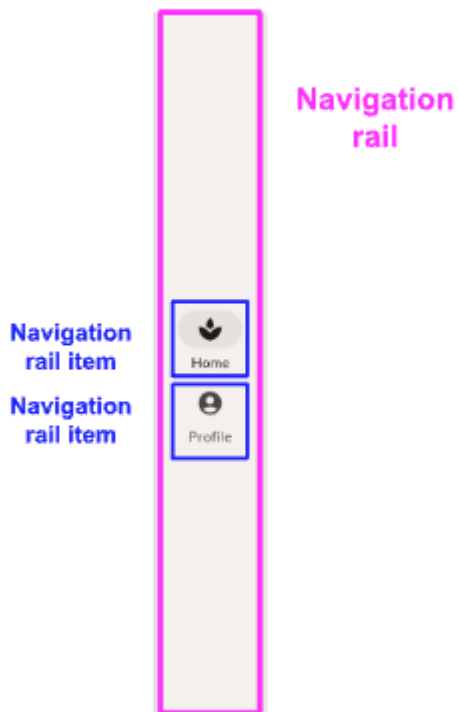
Profile

13. Riel de navegación: Material

Cuando creas diseños para apps, también debes tener en cuenta cómo se verá en varias configuraciones, incluido el modo horizontal de tu teléfono. A continuación, se muestra el diseño de la app en este modo. Observa cómo la navegación inferior se convierte en un riel a la izquierda del contenido de la pantalla.



Para implementar esto, usarás el elemento [NavigationRail](#) componible, que forma parte de la biblioteca de Compose Material y tiene una implementación similar a la de [NavigationBar](#) que se usó para crear la barra de navegación inferior. Dentro del elemento [NavigationRail](#) componible, agregarás elementos [NavigationRailItem](#) para la pantalla principal y el perfil.



Comencemos con la implementación básica de un riel de navegación.

```
import androidx.compose.material3.NavigationRail
import androidx.compose.material3.NavigationRailItem
```

```
@Composable
private fun SootheNavigationRail(modifier: Modifier = Modifier) {
    NavigationRail(
    ){
        Column(
        ){
            NavigationRailItem(
                icon = {
                    Icon(
                        imageVector = Icons.Default.Spa,
                        contentDescription = null
                    )
                },
                label = {
                    Text(stringResource(R.string.bottom_navigation_home))
                }
            )
        }
    }
}
```

```

    },
    selected = true,
    onClick = {}
)

NavigationRailItem(
    icon = {
        Icon(
            imageVector = Icons.Default.AccountCircle,
            contentDescription = null
        )
    },
    label = {
        Text(stringResource(R.string.bottom_navigation_profile))
    },
    selected = false,
    onClick = {}
)
}
}
}

```

NavigationRail...



Home



Profile

Hay algunas adaptaciones de estilo que deberías hacer.

- Agrega 8 dp de padding al principio y al final del riel.
- Para actualizar el color de fondo del riel de navegación, configura su parámetro `containerColor` con el color de fondo del tema de Material. Cuando estableces el color de fondo, el color de los íconos y los textos se adapta automáticamente al color `onBackground` del tema.
- La columna debe rellenar la altura máxima.
- Establece la disposición vertical de la columna como centrada.
- Establece la alineación horizontal de la columna como centrada.
- Agrega 8 dp de padding entre los dos íconos.

La solución final debería ser similar a la siguiente:

```
import androidx.compose.foundation.layout.fillMaxHeight
```

```
@Composable
```

```
private fun SootheNavigationRail(modifier: Modifier = Modifier) {
```

```
    NavigationRail(
```

```
        modifier = modifier.padding(start = 8.dp, end = 8.dp),
```

```
        containerColor = MaterialTheme.colorScheme.background,
```

```
    ){
```

```
        Column(
```

```
            modifier = modifier.fillMaxHeight(),
```

```
            verticalArrangement = Arrangement.Center,
```

```
            horizontalAlignment = Alignment.CenterHorizontally
```

```
        ){
```

```
            NavigationRailItem(
```

```
                icon = {
```

```
                    Icon(
```

```
                        imageVector = Icons.Default.Spa,
```

```
                        contentDescription = null
```

```
                    )
```

```
                },
```

```
                label = {
```

```
                    Text(stringResource(R.string.bottom_navigation_home))
```

```
                },
```

```
        selected = true,
        onClick = {}
    )
    Spacer(modifier = Modifier.height(8.dp))
    NavigationRailItem(
        icon = {
            Icon(
                imageVector = Icons.Default.AccountCircle,
                contentDescription = null
            )
        },
        label = {
            Text(stringResource(R.string.bottom_navigation_profile))
        },
        selected = false,
        onClick = {}
    )
}
}
```

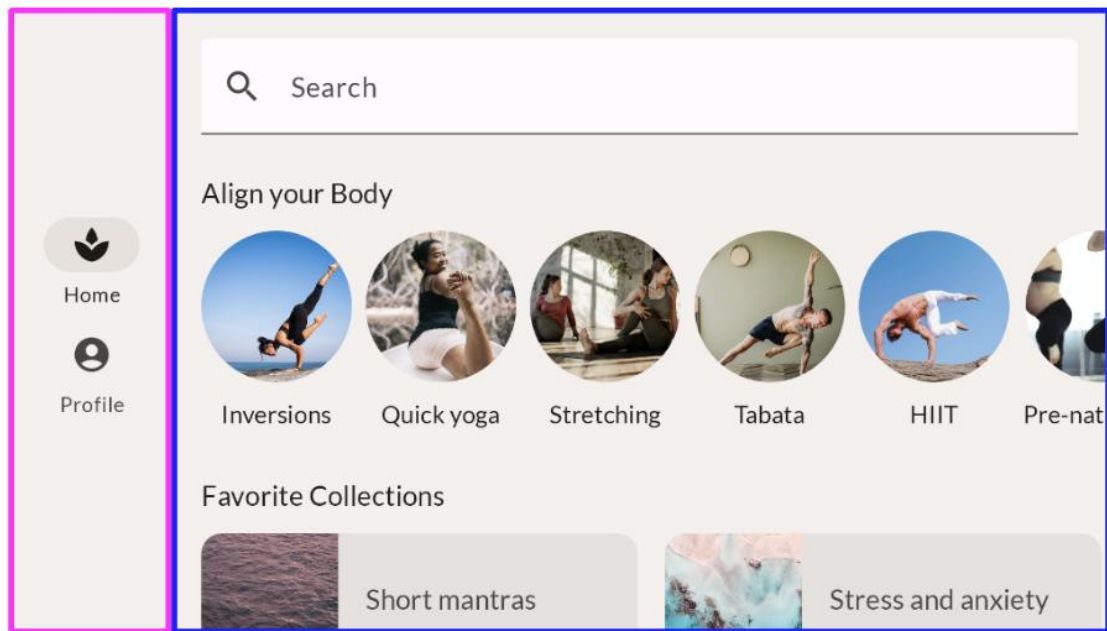


Home



Profile

Ahora, agregaremos el riel de navegación al diseño horizontal.



Navigation rail

Screen Content

Para la versión vertical de la app, usaste Scaffold. Sin embargo, en la vista horizontal, usarás una fila y colocarás el contenido de la pantalla y del riel de navegación uno al lado del otro.

@Composable

```
fun MySootheAppLandscape() {  
    MySootheTheme {  
        Row {  
            SootheNavigationRail()  
            HomeScreen()  
        }  
    }  
}
```

Cuando usaste Scaffold en la versión vertical, este componente también se encargó de configurar el color del contenido de fondo. Para establecer el color del riel de navegación, une la fila en una superficie y configúrala con el color de fondo.

@Composable

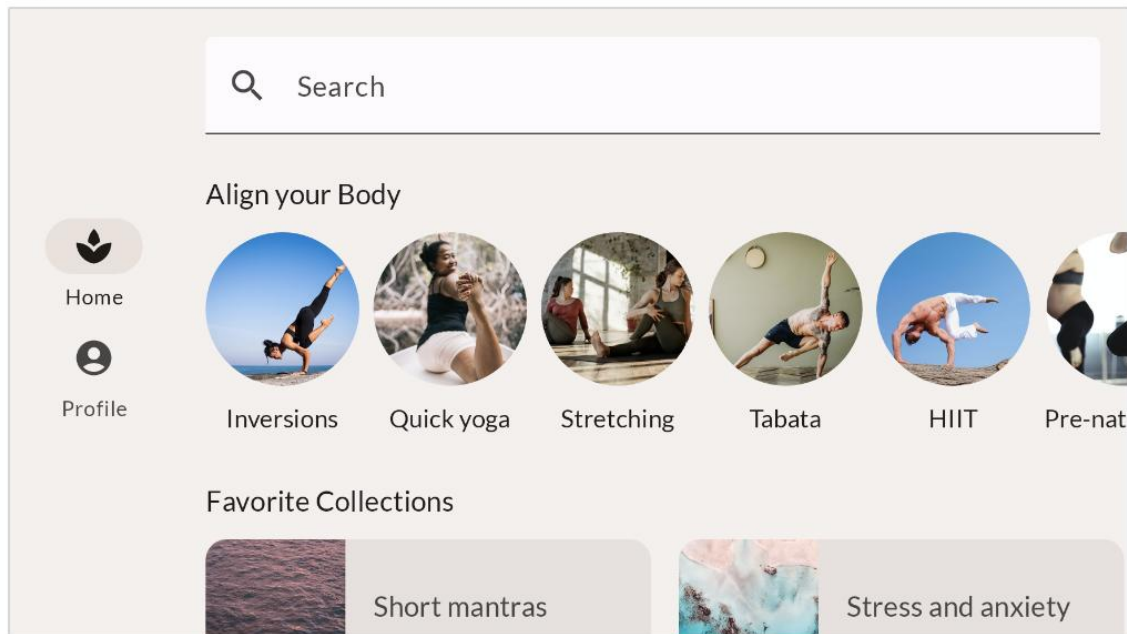
```
fun MySootheAppLandscape() {
```

```

MySootheTheme {
  Surface(color = MaterialTheme.colorScheme.background) {
    Row {
      SootheNavigationRail()
      HomeScreen()
    }
  }
}
}
}

```

MySootheLandscapePreview



14. App de MySoothe: Tamaño de la ventana

Tienes una vista previa del modo horizontal que se ve genial. Sin embargo, si ejecutas la app en un dispositivo o emulador, y lo giras, este no te mostrará la versión horizontal. Esto se debe a que debemos indicarle a la app cuándo mostrar la configuración apropiada. Para ello, usa la función [calculateWindowSizeClass\(\)](#) para ver la configuración actual del teléfono.



Hay tres anchos de clase de tamaño de ventana: compacto, medio y expandido. Cuando la app está en modo vertical, el ancho es el compacto y, cuando está en modo horizontal, el ancho es el expandido. A los fines de este codelab, no trabajarás con el ancho medio.

En el elemento componible `MySootheApp`, actualízalo para que tome el elemento [WindowSizeClass](#) del dispositivo. Si es compacto, pasa la versión vertical de la app. Si es expandido, pasa la versión horizontal de la app.

```
import androidx.compose.material3.windowSizeClass.WindowSizeClass
import androidx.compose.material3.windowSizeClass.WindowWidthSizeClass
@Composable
fun MySootheApp(windowSize: WindowSizeClass) {
    when (windowSize.widthSizeClass) {
        WindowWidthSizeClass.Compact -> {
            MySootheAppPortrait()
        }
        WindowWidthSizeClass.Expanded -> {
            MySootheAppLandscape()
        }
    }
}
```

En `setContent()`, crea un valor llamado `windowSizeClass` establecido en `calculateWindowSize()` y pásalo a `MySootheApp()`.

Dado que `calculateWindowSize()` aún es experimental, deberás habilitar la clase `ExperimentalMaterial3WindowSizeClassApi`.

```
import
androidx.compose.material3.windowclass.ExperimentalMaterial3WindowSizeCla
ssApi
import androidx.compose.material3.windowclass.calculateWindowSizeClass

class MainActivity : ComponentActivity() {
    @OptIn(ExperimentalMaterial3WindowSizeClassApi::class)
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            val windowSizeClass = calculateWindowSizeClass(this)
            MySootheApp(windowSizeClass)
        }
    }
}
```

Ahora, ejecuta la app en tu emulador o dispositivo y observa cómo cambia la pantalla tras una rotación.

12:00

 Search

Align your Body



Inversions



Quick yoga



Stretching



Tabata

Favorite Collections



Short mantras



Stre



Nature meditations



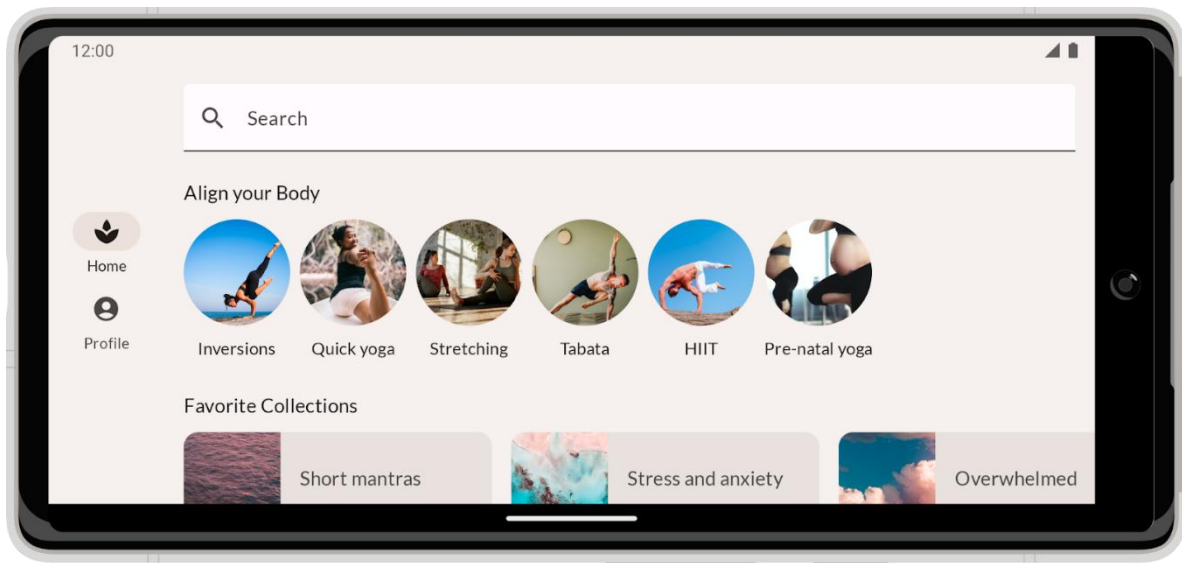
Self



Home



Profile



Referencias:

<https://developer.android.com/develop/ui/compose/modifiers?hl=es-419>